

32-bit Integer Overflow Fix in `ggml` Memory Allocation

1. Problem Analysis (The Math)

The size of a tensor in `ggml` is computed as:

$$S = \text{row_size} \times \prod_{i=1}^{n-1} ne_i$$

On 32-bit platforms, `size_t` is an unsigned 32-bit integer. The maximum representable value is $2^{32} - 1 \approx 4 \text{ GiB}$. When the product S exceeds this limit, the lower 32 bits are retained – a classic **integer wraparound** modulo 2^{32} :

$$S_{\text{truncated}} = S \pmod{2^{32}}$$

This causes the allocator to request a tiny or arbitrary amount of memory instead of the real size, leading to silent memory corruption or a confusing “insufficient memory” error (e.g. 3072 MB for a 300 MB model).

Root cause: The accumulator `data_size` was declared as `size_t`, so every multiplication `data_size *= ne[i]` truncated the intermediate 64-bit result to 32 bits.

2. Mitigation Strategy

Type Promotion

By changing the accumulator to `uint64_t`, all intermediate multiplications are performed in 64-bit arithmetic:

$$S_{\text{uint64}} \in [0, 2^{64} - 1]$$

No information is lost, regardless of the tensor dimensions.

Clamping for Memory Safety

The final allocation must go through `size_t`. We clamp the result to `SIZE_MAX`:

$$\text{alloc_size} = \min(S_{\text{uint64}}, \text{SIZE_MAX})$$

- On 64-bit systems, `SIZE_MAX` is huge ($2^{64} - 1$), so clamping never triggers.
- On 32-bit systems, any tensor exceeding 4 GiB produces a clean, deterministic allocation failure instead of undefined behaviour.

3. Patch (Line-by-Line)

Edit `ggml/src/ggml.c`, function `ggml_new_tensor_impl`.

Before:

```

size_t data_size = ggml_row_size(type, ne[0]);
for (int i = 1; i < n_dims; i++) {
    data_size *= ne[i];
}
...
size_t obj_alloc_size = 0;
if (view_src == NULL && !ctx->no_alloc) {
    obj_alloc_size = data_size;
}

```

After:

```

uint64_t data_size = ggml_row_size(type, ne[0]);
for (int i = 1; i < n_dims; i++) {
    data_size *= ne[i];
}
...
size_t obj_alloc_size = 0;
if (view_src == NULL && !ctx->no_alloc) {
    obj_alloc_size = (data_size <= SIZE_MAX) ? (size_t)data_size : SIZE_MAX;
}

```

Only **two lines** change (the type declaration and the assignment).

4. Validation with Stress Test

A standalone test (`include/ggml_stress_test.c`) verifies the arithmetic **without any model**.

Compile and run from the project root:

```
gcc include/ggml_stress_test.c -o stress_test && ./stress_test
```

Expected output (32-bit system):

- Legacy: `OVERFLOW! The value is incorrect (wraparound).`
- Patched: `Overflow detected => clamped to SIZE_MAX.`
- Final: `✓ The patch logic works correctly.`

On 64-bit systems: Both paths show the same correct size; the verdict is still positive, confirming no side effects.

5. Rebuild and Test with Real Model

1. Rebuild `llama.cpp` :

```
make clean && make -j$(nproc)
```

2. Run inference with the previously failing model:

```
./llama-cli -m path/to/model.gguf -p "Hello" -n 16
```

3. Verify:

- The 3072 MB error is gone.

- RAM usage stays within physical limits (`top` or `free -h`).
 - If the true memory requirement exceeds available RAM, a clean `insufficient memory` error now appears (e.g., 4096 MB), instead of a silent wraparound.
-